COSC2637 Assignment 2 - Task 2 source code (s4271512)

===== task2.pig =====

```
-- Task 2 - Gold vs Total Medals: Winter Olympics Ranking Analysis (COSC2637)
-- Run:    pig -x mapreduce -param START_YEAR=2010 -param END_YEAR=2020 task2.pig
-- Input:  HDFS /medal_table.csv, /games.csv, /countries.csv (comma separated)
-- Output: HDFS /Output/task2 - one pretty-printed block per Games, year ASC.
-- population.csv is not needed for this task and is therefore not loaded.

REGISTER 'task2.py' USING jython AS task2_udf;

-- Year range from the command line (inclusive). Without a -param value the
-- open bounds below select every Games, so no Olympic year is hard coded.
%default START_YEAR 0
%default END_YEAR 9999

-- Remove the previous result so the script can be re-run.
rmf /Output/task2

-- 1. Medal rows: filter to the requested Games first, then keep only the
--    fields needed later (total_medals = gold + silver + bronze).
--    TRIM removes spaces and a Windows carriage return from text fields;
--    bronze is the last field of each CSV line.
medals = LOAD '/medal_table.csv' USING PigStorage(',')
         AS (year:int, country_code:chararray, gold:int, silver:int, bronze:chararray);
medals_in_range = FILTER medals BY year >= $START_YEAR AND year <= $END_YEAR;
medal_counts = FOREACH medals_in_range GENERATE
               year,
               TRIM(country_code) AS country_code,
               gold,
               gold + silver + (int)TRIM(bronze) AS total_medals;

-- 2. Host city of each Games in the same year range (filtered before joining).
games = LOAD '/games.csv' USING PigStorage(',')
        AS (year:int, host_city:chararray, host_country:chararray);
games_in_range = FILTER games BY year >= $START_YEAR AND year <= $END_YEAR;
host_cities = FOREACH games_in_range GENERATE year, TRIM(host_city) AS host_city;

-- 3. Country code -> full country name, exactly as given in countries.csv
--    (e.g. RUS -> Russia / ROC / OAR). The region field is not needed.
countries = LOAD '/countries.csv' USING PigStorage(',')
            AS (country_code:chararray, country_name:chararray, region:chararray);
country_names = FOREACH countries GENERATE
                TRIM(country_code) AS country_code,
                TRIM(country_name) AS country_name;

-- 4. Attach the country name to each medal row. countries is a small lookup
--    table, so a replicated (map-side) join avoids a shuffle.
medals_named = JOIN medal_counts BY country_code, country_names BY country_code USING 'replicated';
medal_rows = FOREACH medals_named GENERATE
             medal_counts::year AS year,
             medal_counts::country_code AS country_code,
             country_names::country_name AS country_name,
             medal_counts::gold AS gold,
             medal_counts::total_medals AS total_medals;

-- 5. One group per Games. Inside each group keep the top three countries by
--    gold medals and by total medals (ties: country_code ascending), so at
--    most six small tuples per Games are passed to the UDF.
medals_by_year = GROUP medal_rows BY year;
leaders = FOREACH medals_by_year {
    gold_sorted  = ORDER medal_rows BY gold DESC, country_code ASC;
    gold_top     = LIMIT gold_sorted 3;
    total_sorted = ORDER medal_rows BY total_medals DESC, country_code ASC;
    total_top    = LIMIT total_sorted 3;
    GENERATE group AS year,
             gold_top.(country_code, country_name, gold) AS top_gold,
             total_top.(country_code, country_name, total_medals) AS top_total;
};

-- 6. Add the host city after the aggregation: one row per Games is joined
--    instead of one row per medal record.
leaders_hosted = JOIN leaders BY year, host_cities BY year USING 'replicated';

-- 7. The Jython UDF assigns the rank numbers and pretty-prints each Games
--    block; the blocks are ordered by year and only the report text is stored.
```

```
reports = FOREACH leaders_hosted GENERATE
          leaders::year AS year,
          task2_udf.format_games_report(leaders::year, host_cities::host_city,
                                        leaders::top_gold, leaders::top_total) AS report;
reports_sorted = ORDER reports BY year ASC;
report_text = FOREACH reports_sorted GENERATE report;
STORE report_text INTO '/Output/task2' USING PigStorage();
```

===== task2.py =====

```
# Task 2 - Gold vs Total Medals: Winter Olympics Ranking Analysis
# Jython UDF (Jython 2.7) registered by task2.pig as task2_udf.
#
# task2.pig groups the medal rows by Olympic year and passes the top three
# candidates of each ranking as a bag of (country_code, country_name, medals)
# tuples. This UDF assigns the rank numbers and builds the pretty-printed
# report block for one Winter Olympic Games:
#
#   2026 Milan-Cortina
#   By Gold Medals:
#   1. Norway (18) | 2. United States (12) | 3. Italy (10)
#   By Total Medals:
#   1. Norway (41) | 2. United States (33) | 3. Italy (30)

TOP_N = 3  # number of leading countries reported for each ranking


def _ranked_line(entries):
    """Rank (country_code, country_name, medals) tuples and format one line.

    Ranking rule: more medals ranks higher; ties are broken by country_code
    in ascending order. Example: in 2018 Germany and Norway both won 14 gold
    medals, so GER ranks ahead of NOR (yay, Germany #1 :D).
    Pig bags are unordered, so the tuples are sorted here even though
    task2.pig already selected them with the same rule.
    """
    ranked = sorted(entries, key=lambda entry: (-entry[2], entry[0]))[:TOP_N]
    return ' | '.join('%d. %s (%d)' % (rank, name, medals)
                      for rank, (code, name, medals) in enumerate(ranked, 1))


@outputSchema('report:chararray')
def format_games_report(year, host_city, top_gold, top_total):
    """Return the report block for one Games.

    The block ends with a newline; together with the record separator that
    PigStorage writes, this leaves one blank line between consecutive Games.
    """
    lines = ['%d %s' % (year, host_city),
             'By Gold Medals:',
             _ranked_line(top_gold),
             'By Total Medals:',
             _ranked_line(top_total)]
    return '\n'.join(lines) + '\n'
```